



UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
BACHARELADO EM ENGENHARIA DE SOFTWARE
PHABLO TAVARES PAIXÃO

Evolução e Refatoração Full-Stack do Sistema SIPROS

**Segurança, Qualidade e Melhorias de Interface em uma
Fábrica de Software**

Goiânia
2026

PHABLO TAVARES PAIXÃO

Evolução e Refatoração Full-Stack do Sistema SIPROS

Segurança, Qualidade e Melhorias de Interface em uma Fábrica de Software

Trabalho de Conclusão apresentado à Coordenação do Curso de Engenharia de Software do Instituto de Informática da Universidade Federal de Goiás, como requisito parcial para obtenção do título de Bacharel em Engenharia de Software.

Orientadora: Profa. Sofia Larissa da Costa Paiva

Co-Orientador: Prof. Juliano Lopes de Oliveira

Goiânia
2026

Todos os direitos reservados. É proibida a reprodução total ou parcial do trabalho sem autorização da universidade, do autor e do orientador(a).

Phablo Tavares Paixão

Graduando em Engenharia de Software pela Universidade Federal de Goiás (UFG). Durante sua trajetória acadêmica, atuou como desenvolvedor back-end no Centro de Excelência em Empreendedorismo Tecnológico (CETT/UFG) e foi bolsista de pesquisa no Centro de Excelência em Inteligência Artificial (CEIA/UFG). Possui experiência em engenharia de software e desenvolvimento mobile (Flutter) pela Personal Fit. Atualmente, atua como Analista de Inteligência Artificial na Rennova, desenvolvendo soluções de automação de processos e integração de inteligência artificial generativa (LLMs).

Agradecimentos

Agradeço à coordenação da Fábrica de Software pelo apoio e trabalho em conjunto, nos guiando ao longo do semestre e nos proporcionando a inestimável oportunidade de atuar em um projeto real desse calibre.

Agradeço à professora Sofia Larissa da Costa Paiva e ao professor Juliano Lopes de Oliveira pelo apoio imenso e orientação fundamentais nesta jornada.

Agradeço aos meus colegas da Equipe Dourada pela dedicação e pelo grande aprendizado que construímos em conjunto durante o desenvolvimento do SIPROS.

Resumo

Paixão, Phablo Tavares. **Evolução e Refatoração Full-Stack do Sistema SI-PROS**. Goiânia, 2026. 41p. Relatório de Graduação. Bacharelado em Engenharia de Software, Instituto de Informática, Universidade Federal de Goiás.

Contexto: A formação acadêmica em Engenharia de Software requer vivência prática para a consolidação do aprendizado teórico. Atuar em ambientes que simulam a indústria, como uma Fábrica de Software, proporciona uma transição realista, exigindo a adaptação a arquiteturas existentes, a garantia contínua da qualidade do produto e a adoção de protocolos de segurança modernos.

Objetivo: Este trabalho tem como objetivo descrever e analisar a experiência de evolução e refatoração full-stack do Sistema Integrado de Processos Seletivos (SI-PROS) da UFG, destacando a implementação de mecanismos robustos de segurança, melhorias na interface de usuário e a adoção de práticas de garantia da qualidade.

Método: O trabalho foi desenvolvido sob a metodologia de pesquisa-ação no contexto da disciplina de Fábrica de Software no semestre 2026/1. Utilizaram-se métodos ágeis para o planejamento e desenvolvimento iterativo. A execução envolveu processos rigorosos de Verificação e Validação (V&V) entre Casos de Uso e protótipos, além da aplicação sistemática de revisões por pares (*Code Review*) e testes automatizados.

Resultados: A intervenção resultou na integração bem-sucedida da autenticação do Google via Keycloak, adotando a extensão PKCE (*Proof Key for Code Exchange*) para mitigar vulnerabilidades estruturais no *front-end* em Angular. Simultaneamente, as práticas de V&V permitiram a correção de dezenas de inconsistências de arquitetura e usabilidade, enquanto a estruturação formal da suíte de testes de unidade garantiu a integridade do *back-end* contra regressões.

Conclusões: Os resultados evidenciam que a implementação de protocolos de segurança em Aplicações de Página Única (SPAs) exige um rigor arquitetural que afeta todo o fluxo do sistema. A experiência atestou que o emprego constante de práticas de qualidade (testes e V&V) é indispensável em projetos colaborativos. Por fim, a atuação na Fábrica de Software proporcionou um amadurecimento técnico e comportamental inestimável, aproximando genuinamente a vivência acadêmica da realidade do mercado de tecnologia.

Palavras-chave

Engenharia de Software; Fábrica de Software; Autenticação e Segurança; Qualidade de Software; Desenvolvimento Full-Stack.

Abstract

Paixão, Phablo Tavares. **Full-Stack Evolution and Refactoring of the SIPROS System: Security, Quality, and Interface Improvements in a Software Factory**. Goiânia, 2026. 41p. Relatório de Graduação. Bacharelado em Engenharia de Software, Instituto de Informática, Universidade Federal de Goiás.

Context: Academic training in Software Engineering requires practical experience to consolidate theoretical learning. Working in environments that simulate the industry, such as an academic Software Factory, provides a realistic transition, demanding adaptation to existing architectures, continuous quality assurance, and the adoption of modern security protocols.

Objective: This work aims to describe and analyze the practical experience of the full-stack evolution and refactoring of the Integrated Selection Processes System (SIPROS) at UFG, highlighting the implementation of robust security mechanisms, user interface improvements, and the adoption of quality assurance practices.

Method: The work was developed using action research methodology within the context of the Software Factory course during the 2026/1 semester. Agile methods were utilized for planning and iterative development. The execution involved rigorous Verification and Validation (V&V) processes between Use Cases and prototypes, as well as the systematic application of peer reviews (Code Review) and automated testing.

Results: The intervention resulted in the successful integration of Google authentication via Keycloak, adopting the PKCE (Proof Key for Code Exchange) extension to mitigate structural vulnerabilities in the Angular front-end. Simultaneously, the V&V practices allowed the correction of dozens of architectural and usability inconsistencies, while the formal structuring of the unit test suite ensured the back-end's integrity against regressions.

Conclusions: The results show that implementing security protocols in Single Page Applications (SPAs) requires architectural rigor that affects the entire system flow. The experience confirmed that the constant employment of quality practices (testing and V&V) is indispensable in collaborative projects. Finally, working in the Software Factory provided invaluable technical and behavioral maturation, genuinely bringing the academic experience closer to the reality of the technology market.

Keywords

Software Engineering; Software Factory; Authentication and Security; Software Quality; Full-Stack Development.

Sumário

Lista de Figuras	10
Lista de Tabelas	11
Lista de Algoritmos	12
Lista de Códigos de Programas	13
1 Introdução	14
1.1 Motivação e Justificativa	14
1.2 Contexto da Experiência	15
1.3 Objetivo do Trabalho	15
1.3.1 Objetivo Geral	15
1.3.2 Objetivos Específicos	15
1.4 Metodologia do Trabalho	16
1.5 Organização do Trabalho	17
2 Bases Teóricas e Tecnológicas	18
2.1 Tópicos Específicos Trabalhados no TCC	18
2.1.1 Autenticação e Segurança em Aplicações Web	18
2.1.2 Verificação e Validação (V&V)	18
2.1.3 Qualidade e Testes de Software	19
2.2 Processos de Ciclo de Vida de Software	19
2.3 Organização do Trabalho Colaborativo em Software	19
2.4 Ferramentas e Tecnologias Adotadas	20
3 Relato de Experiência	21
3.1 Contextualização do Projeto	21
3.1.1 Ambiente Organizacional	21
3.1.2 Forma de Participação no Projeto	22
3.2 Atividades Realizadas e Resultados Gerados	22
3.3 Integração com a Equipe e Processo de Trabalho	23
3.4 Desafios Enfrentados	23
3.5 Soluções Adotadas e Resultados Obtidos	23
3.6 Avaliação de Qualidade de Software	24
3.7 Aprendizados e Desenvolvimento Profissional	24

4	Conclusões	25
4.1	Considerações Finais	25
4.2	Síntese da Experiência na Fábrica de Software	25
4.3	Trabalhos Relacionados	26
4.4	Trabalhos Futuros	27
	Referências	28
A	Como usar este modelo para elaborar seu TCC	29
A.1	Introdução	29
A.2	Descrição da classe inf-ufg	30
A.2.1	Opções da classe	30
A.2.2	Parâmetros da classe	30
A.2.3	Elementos Pré-Textuais	32
A.3	Elementos do texto	33
A.3.1	Figuras	33
A.3.2	Subfiguras	36
A.3.3	Tabelas	37
A.3.4	Algoritmos	37
A.3.5	Códigos de Programa	38
A.3.6	Teoremas, Corolários e Demonstrações	39
A.3.7	Citações Longas	40
A.3.8	Referências Bibliográficas	40

Lista de Figuras

A.1	Uma figura típica.	35
A.2	Esta figura é um exemplo de um rótulo de figura que ocupa mais de uma linha, devendo ser indentado e justificado.	35
A.3	Figura incluída no texto com a classe <code>graphicx</code> .	36
A.4	(a) e (b) representam dois exemplos do uso de subfiguras dentro de uma única figura.	36
	(b) Segunda subfigura (um pedaço).	36

Lista de Tabelas

A.1 Conteúdo do diretório

38

Lista de Algoritmos

A.1 $MSR(A, i, j)$

38

Lista de Códigos de Programas

A.1 `insertionsort()`

39

Introdução

1.1 Motivação e Justificativa

A sociedade contemporânea apresenta uma dependência crescente em relação a sistemas de software, exigindo que aplicações, especialmente em contextos institucionais e governamentais, sejam desenvolvidas com alto rigor arquitetural e foco em segurança. Plataformas acadêmicas lidam diariamente com dados sensíveis e necessitam de processos eficientes de desenvolvimento e manutenção contínua.

O Sistema Integrado de Processos Seletivos (SIPROS) da Universidade Federal de Goiás (UFG) foi concebido para centralizar e padronizar editais de seleção. Em seu estado prévio ao semestre letivo de 2026/1, o sistema encontrava-se em uma fase de maturação técnica que demandava evoluções estruturais, tais como: a transição de fluxos simulados (*mockados*) para integrações reais, a consolidação do roteamento e a externalização de configurações, além da adoção de uma camada de autenticação alinhada aos padrões contemporâneos de segurança.

Atuar na resolução desses problemas em um ambiente colaborativo é fundamental. O trabalho em equipe em projetos de software reais traz desafios organizacionais e técnicos que não podem ser simulados em projetos acadêmicos teóricos isolados. A vivência e a aplicação de boas práticas de Engenharia de Software, em um cenário legado e em constante evolução, proporcionam um ganho prático substancial. O relato desta experiência justifica-se por sua valiosa contribuição acadêmica e prática, detalhando como problemas críticos de segurança e consistência de interface podem ser resolvidos. Para estudantes e futuras turmas, este documento serve como um guia de lições aprendidas; para a coordenação da Fábrica de Software, documenta a evolução arquitetural do próprio SIPROS.

1.2 Contexto da Experiência

O presente relato se insere no contexto da disciplina de Prática em Engenharia de Software, ofertada pelo Instituto de Informática da UFG durante o semestre letivo de 2026/1. A Fábrica de Software acadêmica tem como propósito simular o ambiente de mercado e propõe aos discentes a manutenção evolutiva e refatoração de sistemas reais.

O autor do trabalho integrou a "Equipe Dourada", composta por um total de sete discentes, responsável por atuar no desenvolvimento *full-stack* do sistema SIPROS. No que tange à organização do processo de desenvolvimento, a equipe não esteve obrigada a adotar de forma estrita nenhum *framework* ágil específico. Em vez disso, adotou-se uma metodologia de desenvolvimento **inspirada** nas metodologias ágeis e no *Scrum*, incorporando suas principais características (como as *Sprints*), mas sendo adaptada continuamente para o contexto e necessidades da equipe ao longo do semestre. A coordenação da Fábrica atuou em um papel semelhante ao de um *Product Owner*, definindo as demandas e prioridades, enquanto os discentes possuíam autonomia para o autogerenciamento, divisão de tarefas e aplicação de ferramentas visuais, como quadros *Kanban*.

1.3 Objetivo do Trabalho

Este Trabalho de Conclusão de Curso possui um objetivo principal que delinea a pesquisa, sendo o mesmo desdobrado em metas específicas que orientam a descrição dos resultados obtidos.

1.3.1 Objetivo Geral

O objetivo principal deste trabalho é descrever e analisar a experiência prática de evolução e refatoração *full-stack* do sistema SIPROS no contexto da Fábrica de Software acadêmica da UFG, com ênfase na implementação de mecanismos robustos de segurança, autenticação e na adoção de práticas visando a elevação da qualidade do software.

1.3.2 Objetivos Específicos

Para alcançar o objetivo geral traçado, foram definidos os seguintes objetivos específicos:

1. Descrever o ambiente organizacional, a metodologia de trabalho inspirada em métodos ágeis e a dinâmica de colaboração adotada no projeto SIPROS.

2. Relatar as implementações técnicas voltadas à segurança da informação, destacando a integração com o provedor de identidade do Google via Keycloak, utilizando a extensão PKCE.
3. Descrever a execução dos processos de Verificação e Validação (V&V) aplicados para garantir a conformidade arquitetural da interface (*front-end*) com os Casos de Uso estabelecidos.
4. Apresentar as práticas de Garantia de Qualidade (QA) adotadas pela equipe, evidenciando a estruturação da suíte de testes automatizados e a adoção obrigatória das revisões por pares (*Code Review*).
5. Refletir criticamente sobre os desafios e gargalos vivenciados, analisando o amadurecimento e a evolução nas *hard skills* e *soft skills* proporcionados por essa vivência de caráter profissional.

1.4 Metodologia do Trabalho

No âmbito metodológico acadêmico, a elaboração do presente trabalho baseia-se na metodologia de **pesquisa-ação**. Diferentemente de um estudo de caso puramente observacional, a pesquisa-ação caracteriza-se pela intervenção ativa do pesquisador no próprio ambiente para a resolução de um problema real de Engenharia de Software. Neste cenário, o autor atuou ativamente como membro da equipe de desenvolvimento, mapeando inconsistências, projetando soluções de *design* arquitetural e efetuando modificações no código-fonte, participando diretamente da resolução do problema do sistema legado e na consequente implantação de um padrão de segurança seguro e atual.

Para fundamentar as atividades realizadas, a pesquisa se apoiou em literatura técnico-científica reconhecida na área, como o corpo de conhecimento da Engenharia de Software (SWEBOK) [Bourque & Fairley, 2014] para os conceitos de Verificação e Validação, e normas internacionais como a ISO/IEC 25010 [ISO/IEC 25010:2011, 2011] para modelagem de qualidade de produto. Complementarmente, consultou-se a documentação técnica pertinente (RFCs do protocolo OAuth 2.0 [Hardt, 2012, Sakimura et al., 2015] e manuais oficiais do *Keycloak* [Red Hat, 2026] e do ecossistema *Angular* [Google, 2026]) e utilizou-se do acervo de artefatos produzidos internamente pelo projeto (*Issues*, *Pull Requests* e documentação de Casos de Uso).

Ressalta-se que a metodologia de desenvolvimento de software utilizada no dia a dia da equipe — inspirada no paradigma ágil — distingue-se desta metodologia acadêmica de pesquisa-ação e será tratada em maiores detalhes no Capítulo 3.

1.5 Organização do Trabalho

Para organizar o fluxo lógico do documento e guiar a leitura do relato, este trabalho encontra-se estruturado da seguinte forma:

- Capítulo 2: Apresenta as bases teóricas, conceituais e tecnológicas requeridas na execução das tarefas, cobrindo tópicos de Qualidade de Software, Verificação e Validação, além das premissas de Segurança em Aplicações Web baseadas em tokens (OAuth 2.0 e PKCE).
- Capítulo 3: Concentra a descrição detalhada do relato de experiência do discente. Expõe os desafios enfrentados no *onboarding*, as tomadas de decisões arquiteturais na integração com o GCP (*Google Cloud Platform*), o rastreo e mitigação de defeitos e os resultados consolidados na melhoria do SIPROS.
- Capítulo 4: Corresponde às considerações finais, contendo a reflexão crítica do autor sobre a evolução das competências necessárias ao Engenheiro de Software, um panorama dos Trabalhos Futuros para o projeto e as percepções consolidadas a partir do semestre na Fábrica de Software.

Bases Teóricas e Tecnológicas

Este capítulo descreve os pilares teóricos e as tecnologias que fundamentaram as atividades de evolução e refatoração do SIPROS. Apresenta-se o embasamento em segurança para aplicações web, práticas de Verificação e Validação (V&V), processos de evolução de software e metodologias de trabalho colaborativo, além da análise crítica das principais ferramentas adotadas.

2.1 Tópicos Específicos Trabalhados no TCC

O desenvolvimento e a manutenção do SIPROS envolveram três eixos técnicos centrais da Engenharia de Software: segurança, validação de interface e qualidade de código.

2.1.1 Autenticação e Segurança em Aplicações Web

A autenticação em Aplicações de Página Única (SPAs), como aquelas desenvolvidas em Angular, apresenta desafios de segurança específicos. A literatura atual [Hardt, 2012] desencoraja o uso do *Implicit Flow* para OAuth 2.0 devido ao risco de vazamento de *tokens* no histórico do navegador ou por ataques de interceptação (CSRF).

Para mitigar essas vulnerabilidades, o padrão ouro atual é o *Authorization Code Flow* com a extensão PKCE (*Proof Key for Code Exchange*) [Sakimura et al., 2015]. O PKCE substitui a necessidade de um *client secret* estático por um verificador criptográfico dinâmico gerado a cada requisição, garantindo segurança robusta na delegação de autorização sem expor credenciais sensíveis no lado do cliente.

2.1.2 Verificação e Validação (V&V)

Segundo o *Software Engineering Body of Knowledge* (SWEBOK) [Bourque & Fairley, 2014] e a norma IEEE 1012, os processos de Verificação e Validação (V&V) visam assegurar que o software seja construído corretamente e atenda

aos requisitos definidos. A Verificação assegura a conformidade do sistema com suas especificações técnicas (ex: protótipos em Figma vs. Casos de Uso), enquanto a Validação garante que a solução atende à real necessidade do usuário final. No ciclo de vida do projeto, a aplicação sistemática de V&V precocemente impede que inconsistências de *design* ou requisitos conflitantes avancem para as fases de produção.

2.1.3 Qualidade e Testes de Software

A qualidade do software pode ser definida por atributos como adequação funcional, manutenibilidade e confiabilidade, conforme a norma internacional ISO/IEC 25010 [ISO/IEC 25010:2011, 2011]. A automação de testes de unidade é um pilar prático para sustentar essa qualidade. Testes adequadamente isolados e rastreáveis garantem que novas implementações e refatorações arquiteturais ocorram sem causar regressões, reduzindo significativamente o débito técnico ao longo do tempo.

2.2 Processos de Ciclo de Vida de Software

A evolução do SIPROS inseriu-se primariamente nos processos de **Manutenção Evolutiva** e de **Garantia da Qualidade**. A manutenção de um sistema legado demanda um ciclo iterativo de análise, planejamento, modificação e validação.

Neste contexto, o processo de V&V integra-se a todas as fases do ciclo de vida, atuando como um filtro de qualidade. Ao estabelecer a "fonte da verdade" (Casos de Uso) antes de programar as telas, o processo impede a propagação de defeitos das fases de engenharia de requisitos para o código-fonte, reduzindo custos de retrabalho e instabilidades operacionais.

2.3 Organização do Trabalho Colaborativo em Software

No desenvolvimento contemporâneo, a agilidade na entrega de valor é essencial. As metodologias ágeis (como Scrum e Kanban) [Schwaber & Sutherland, 2020] fornecem um arcabouço para desenvolvimento iterativo, divisão de tarefas e entregas contínuas, mantendo o foco na adaptação a mudanças em vez do planejamento engessado.

Nesse modelo colaborativo, a comunicação clara e o compartilhamento de responsabilidades são cruciais. Práticas como a **Revisão de Código (Code Review)** funcionam tanto como uma barreira técnica para detectar defeitos antes da integração, quanto como uma ferramenta gerencial para nivelar o conhecimento da equipe sobre a base de código, garantindo a coesão da arquitetura e o cumprimento dos padrões estabelecidos pelo projeto.

2.4 Ferramentas e Tecnologias Adotadas

As atividades de Engenharia de Software no SIPROS foram suportadas por um ecossistema de ferramentas sólidas da indústria, cujas escolhas basearam-se na aderência à arquitetura pré-estabelecida do projeto.

- **Keycloak:** Trata-se de uma solução robusta em código aberto [Red Hat, 2026] para gerenciamento de identidades e acessos. Seu principal ponto forte é a segurança avançada e a facilidade de integração via OAuth 2.0. Contudo, apresenta a limitação de uma curva de aprendizado íngreme em sua configuração inicial, exigindo alto conhecimento sobre protocolos de rede e gestão de variáveis de ambiente.
- **Angular:** Utilizado como *framework front-end* [Google, 2026], destaca-se pela sua arquitetura componentizada, viabilizando o desenvolvimento de interfaces altamente fiéis aos protótipos visuais. Sua principal limitação é a rigidez estrutural e a dependência do *TypeScript*, que impõem barreiras de entrada maiores para desenvolvedores recém-integrados.
- **Docker:** Fundamental para o isolamento das dependências e padronização do ambiente local de desenvolvimento. Como desvantagem, exige um elevado custo computacional (*overhead*), o que pode causar instabilidades e lentidão em máquinas de desenvolvimento com capacidades de *hardware* mais restritas.
- **GitHub:** Plataforma consolidada para controle de versão baseado no *Git*. Além do versionamento do código-fonte, provou-se indispensável no gerenciamento do fluxo de trabalho por meio do rastreamento de problemas (*Issues*) e no suporte direto à cultura de revisão de código (*Pull Requests*).

A conjunção dessas tecnologias propiciou as condições técnicas necessárias para sustentar as práticas metodológicas da equipe, viabilizando as refatorações discutidas no Capítulo 3.

Relato de Experiência

3.1 Contextualização do Projeto

O Sistema Integrado de Processos Seletivos (SIPROS) é uma plataforma institucional da Universidade Federal de Goiás (UFG), projetada para centralizar a criação e o acompanhamento de editais. A criticidade desse sistema é alta, uma vez que ele transaciona dados sensíveis de candidatos e gerencia documentos oficiais da instituição.

No momento em que o projeto foi assumido pela equipe, o software encontrava-se em uma fase de desenvolvimento na qual a arquitetura de segurança e a integração de componentes estavam em transição. Para fins de prototipação inicial, o sistema utilizava credenciais inseridas provisoriamente no código (*hardcoded*), a comunicação entre o *front-end* e o *back-end* ainda necessitava de consolidação estrutural, e alguns fluxos de acesso eram simulados (*mocks*). O principal objetivo do projeto neste semestre consistiu em evoluir essa base arquitetural, substituindo as abordagens provisórias por implementações definitivas e estabelecendo um fluxo de autenticação robusto.

3.1.1 Ambiente Organizacional

O relato ocorreu no contexto da disciplina de Prática em Engenharia de Software (Fábrica de Software), cujo objetivo é proporcionar vivência realista na manutenção e evolução de um sistema de grande porte. A equipe responsável, denominada "Equipe Dourada", foi composta por sete discentes. Destes, Ester e Lucas atuaram predominantemente na comunicação técnica com a coordenação, que por sua vez assumiu o papel semelhante ao de um *Product Owner* (PO).

Para o gerenciamento do processo de desenvolvimento, a equipe optou por uma metodologia flexível inspirada no *Scrum* e no *Kanban*. O trabalho foi organizado em iterações regulares (*Sprints*) de duas semanas, ao fim das quais ocorriam reuniões de revisão diretamente com a coordenação para a validação das entregas. As principais tecnologias e ferramentas empregadas englobaram o Angular para o *front-end*, Keycloak para o gerenciamento de identidades, Docker para o encapsulamento do ambiente, Figma

para o *design* de interfaces e a plataforma GitHub para versionamento de código e gestão das tarefas.

3.1.2 Forma de Participação no Projeto

O autor do trabalho ingressou no projeto exercendo o papel de desenvolvedor *full-stack*. A integração ao ambiente tecnológico ocorreu por meio de um *onboarding* estruturado pela própria Fábrica, cujo foco foi o nivelamento prático em controle de versão no Git e na orquestração de contêineres via Docker. Essa familiarização inicial garantiu que todo o ambiente de desenvolvimento local operasse de maneira isolada e padronizada, permitindo uma transição fluida para as atividades de codificação do sistema legado.

3.2 Atividades Realizadas e Resultados Gerados

As atividades desenvolvidas abrangeram a intervenção simultânea em três frentes principais: segurança, interface e testes automatizados. A seguir, detalham-se as principais contribuições técnicas executadas pelo autor.

A primeira frente consistiu na refatoração da camada de segurança (*Issue* #214 e #253). O desenvolvimento exigiu a substituição do fluxo estático pela integração com o protocolo OAuth 2.0 via Keycloak, empregando especificamente o fluxo *Authorization Code* acoplado à extensão PKCE. A adoção desse padrão mitigou os riscos inerentes a aplicações de página única (SPA), protegendo o processo de delegação de acesso contra a interceptação de *tokens*. Adicionalmente, implementou-se o redirecionamento automático para a autenticação centralizada do Google, otimizando a experiência do usuário final.

A segunda frente pautou-se nos processos de Verificação e Validação de interface (*Issue* #139 e #179). A atividade focou no levantamento de divergências estruturais entre a documentação oficial (Casos de Uso) e os protótipos de alta fidelidade desenhados no Figma, resultando em correções aplicadas diretamente nos componentes visuais desenvolvidos em Angular.

A terceira frente atuou na Garantia da Qualidade (QA). O autor estruturou o ambiente básico da suíte de testes de unidade (*Issue* #304). O propósito dessa implementação foi prevenir que as refatorações arquiteturais resultassem em regressões nas regras de negócio da aplicação.

3.3 Integração com a Equipe e Processo de Trabalho

A dinâmica de trabalho colaborativo pautou-se na autonomia técnica e na comunicação proativa. Os alinhamentos diários ou de acompanhamento foram centralizados no Discord (ferramenta oficial para chamadas de voz e revisões conjuntas) e em grupos no WhatsApp (para comunicação ágil).

O processo de trabalho da equipe exigiu um gerenciamento cuidadoso para lidar com conflitos de código. Essa coordenação foi feita estabelecendo a clareza sobre qual ramificação (*branch*) cada desenvolvedor possuía responsabilidade em dado momento, mitigando sobreposições no repositório. O processo exigia, por definição, que qualquer alteração proposta (*Pull Request*) passasse pelo fluxo estruturado de revisão antes de ser integrada à aplicação principal.

3.4 Desafios Enfrentados

Os desafios apresentados possuíam naturezas tanto técnicas quanto de engenharia de requisitos. Do ponto de vista técnico, o *onboarding* na configuração do Keycloak exigiu aprendizado sobre os escopos e as diretrizes do *Google Cloud Platform* (GCP). Houve também complexidade substancial na manutenção dos estados do aplicativo Angular, devido à perda de contexto gerada pelos redirecionamentos inerentes ao fluxo de autenticação externo.

Quanto aos desafios organizacionais, a principal dificuldade encontrada referiu-se à inexistência de uma "fonte da verdade"unificada. Os desenvolvedores debatiam-se constantemente frente à contradição entre os requisitos técnicos descritos nos modelos textuais de Casos de Uso e o que estava desenhado nos protótipos visuais no Figma.

3.5 Soluções Adotadas e Resultados Obtidos

Para solucionar os impasses organizacionais relativos aos requisitos, a equipe deliberou e oficializou que os documentos de Casos de Uso funcionariam como a única fonte da verdade em caso de divergência com os protótipos visuais.

No âmbito técnico, a substituição definitiva das telas estáticas e da autenticação em *mock* pela implementação do fluxo OAuth/PKCE trouxe maturidade ao sistema. A aplicação das correções de V&V erradicou diversas inconsistências visuais, e a infraestrutura do repositório foi higienizada com a migração das credenciais embutidas no código para variáveis de ambiente isoladas. Como resultado prático consolidado, o SIPROS evoluiu de um estado acadêmico experimental para uma arquitetura de *software* aderente às diretrizes de segurança de mercado.

3.6 Avaliação de Qualidade de Software

A estratégia de qualidade ancorou-se em dois eixos paralelos de validação. O primeiro envolveu a estruturação de uma fundação formal para testes de unidade automatizados voltados ao *back-end*, provendo a segurança indispensável para a evolução orgânica do código. O segundo eixo foi ancorado fortemente no fator humano por meio do processo obrigatório de *Code Review*. Estabeleceu-se como norma inegociável que nenhuma alteração técnica seria introduzida na ramificação principal sem o escrutínio e aprovação por pares, assegurando aderência aos padrões de código preestabelecidos.

3.7 Aprendizados e Desenvolvimento Profissional

A vivência na Fábrica de Software extrapolou o aprendizado estritamente acadêmico ao submeter o autor a responsabilidades genuínas exigidas pelo mercado de trabalho. No espectro das *hard skills*, a experiência solidificou o domínio de protocolos modernos de autenticação (GCP, Keycloak, OAuth 2.0), além da consolidação prática no desenvolvimento voltado para o ecossistema Angular e na orquestração de contêineres Docker.

No tocante às *soft skills*, houve uma evolução marcante na capacidade de autogerenciamento, na resolução autônoma de bloqueios operacionais e na proficiência em debater arquitetura técnica com outros profissionais. Ao final do projeto, o discente consolidou a visão de que atuar na manutenção e refatoração crítica de uma aplicação preexistente confere um peso e um realismo profissional incomensuráveis, complementando sua formação integral como Engenheiro de Software.

Conclusões

4.1 Considerações Finais

O presente trabalho teve como motivação relatar a experiência prática de evolução e refatoração do Sistema Integrado de Processos Seletivos (SIPROS) no escopo da Fábrica de Software da Universidade Federal de Goiás. A metodologia adotada fundamentou-se na pesquisa-ação e em processos inspirados em abordagens ágeis, garantindo adaptação contínua e entregas incrementais.

A análise da vivência demonstra que o objetivo geral foi plenamente alcançado: a equipe implementou com sucesso os mecanismos de segurança necessários (Keycloak e PKCE) e consolidou práticas robustas de garantia da qualidade (QA) no *front-end* e *back-end*. De forma análoga, os objetivos específicos foram concretizados por meio do mapeamento de V&V da interface, da adoção sistemática de *Code Review* e da reestruturação da base de testes automatizados, erradicando abordagens provisórias herdadas e entregando um sistema arquiteturalmente mais maduro.

4.2 Síntese da Experiência na Fábrica de Software

A imersão na Fábrica de Software revelou uma disparidade substancial entre projetos estritamente acadêmicos e a atuação em um sistema de criticidade real. A responsabilidade exigida ao transacionar dados sensíveis e gerenciar a infraestrutura de uma aplicação com impacto institucional exigiu da equipe um nível de profissionalismo inerente ao mercado de trabalho, moldando positivamente a postura técnica e ética dos discentes envolvidos.

No que tange aos aprendizados, a experiência proporcionou uma evolução evidente tanto em competências técnicas (*hard skills*) quanto comportamentais (*soft skills*). Houve grande consolidação no domínio do *framework* Angular, na orquestração de contêineres via Docker e na integração de segurança com a *Google Cloud Platform* (GCP) e OAuth 2.0. Sob a ótica comportamental, o projeto instigou aprimoramentos no autogerenciamento.

ciamento, na comunicação interativa com os pares e na autonomia para defender e aplicar decisões arquiteturais diante de problemas complexos.

As principais dificuldades enfrentadas concentraram-se na curva inicial de aprendizado para a configuração de infraestrutura (*Keycloak*) e na ausência temporária de uma "fonte da verdade"unificada para os requisitos de *front-end*.

Como lições aprendidas e sugestões para os estudantes que vivenciarão a Fábrica de Software em semestres futuros, recomenda-se fortemente:

1. **Comunicação Proativa e Cultura de Code Review:** Em ambientes colaborativos, a transparência sobre qual funcionalidade está sendo desenvolvida e a disposição para revisar o código dos colegas são essenciais para evitar conflitos no repositório. O processo de revisão deve ser encarado não como um obstáculo, mas como a principal ferramenta de disseminação de conhecimento.
2. **Gestão Ativa de Débito Técnico e Código Legado:** Não tenham receio de atuar em bases de código legadas. O primeiro instinto de um desenvolvedor iniciante pode ser reescrever todos os componentes do zero, mas a verdadeira maturidade em Engenharia de Software demonstra-se na habilidade de analisar, compreender o legado e refatorá-lo de forma incremental e segura.
3. **Priorização da Garantia de Qualidade:** A cultura de testes de unidade nunca deve ser tratada como uma etapa secundária ou deixada para o final. Valorizem a escrita e a automação de testes desde o primeiro dia da *Sprint*, garantindo que a evolução do software não se torne refém de regressões ocultas no futuro.
4. **Definição de uma Fonte da Verdade:** Sempre certifiquem-se de alinhar os requisitos técnicos com os documentos formais (como os Casos de Uso) antes de iniciar a prototipação e o desenvolvimento no *front-end*. O alinhamento precoce previne o desperdício de tempo decorrente de inúmeros retrabalhos de interface.

4.3 Trabalhos Relacionados

O relato descrito corrobora com a literatura existente no campo de Engenharia de Software Empírica, particularmente naqueles trabalhos que documentam os desafios na adoção de metodologias inspiradas em práticas ágeis em ambientes educacionais controlados. As observações sobre a necessidade de rigor na transição de fluxos de autenticação em estágio de prototipação para implementações maduras baseadas no protocolo OAuth 2.0 com PKCE também encontram respaldo em estudos de caso contemporâneos sobre Segurança em Aplicações de Página Única (SPAs). Observa-se que a adoção de práticas sólidas de V&V, testes automatizados e *Code Review* continua sendo apontada pela literatura como a estratégia mais eficaz de mitigação de débito técnico orgânico.

4.4 Trabalhos Futuros

Apesar da evolução estrutural promovida neste ciclo, o SIPROS apresenta oportunidades contínuas para aperfeiçoamento. Diante das limitações temporais do semestre e do escopo do projeto, sugerem-se os seguintes trabalhos futuros:

- **Testes de Ponta a Ponta (*End-to-End*):** Iniciar a implementação de testes E2E no *front-end* utilizando ferramentas como *Cypress*, a fim de simular e validar o fluxo completo do usuário, complementando a suíte de testes de unidade já existente.
- **Expansão da Cobertura do *Back-end*:** Dar seguimento à estruturação das rotinas de QA, visando atingir uma cobertura abrangente nos serviços e controladores de maior criticidade da aplicação.
- **Conclusão do V&V de Interface:** Finalizar o mapeamento de Verificação e Validação para os componentes visuais e telas secundárias remanescentes, garantindo a paridade total de todos os módulos com os modelos textuais de Casos de Uso.
- **Otimização do Ambiente Local:** Avaliar a reestruturação dos contêineres Docker e a carga de serviços simulados localmente no repositório, com o intuito de reduzir o elevado consumo de recursos de *hardware* (*overhead*) nas máquinas de desenvolvimento pessoal.

Referências

- [Bourque & Fairley, 2014] Bourque, P. & Fairley, R. E. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0*. Washington, DC, USA: IEEE Computer Society Press.
- [Google, 2026] Google (2026). Angular - the modern web developer's platform. <https://angular.dev/>. Acesso em: 19 jun. 2026.
- [Hardt, 2012] Hardt, D. (2012). *The OAuth 2.0 Authorization Framework*. RFC 6749, RFC Editor.
- [ISO/IEC 25010:2011, 2011] ISO/IEC 25010:2011 (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Standard, International Organization for Standardization.
- [Red Hat, 2026] Red Hat (2026). Keycloak documentation. <https://www.keycloak.org/documentation>. Acesso em: 19 jun. 2026.
- [Sakimura et al., 2015] Sakimura, N., Bradley, J., & Agarwal, N. (2015). *Proof Key for Code Exchange by OAuth Public Clients*. RFC 7636, RFC Editor.
- [Schwaber & Sutherland, 2020] Schwaber, K. & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org.

Como usar este modelo para elaborar seu TCC

A.1 Introdução

$\text{\LaTeX}$ é um sistema de editoração eletrônica muito usado para produzir documentos científicos de alta qualidade tipográfica. Este apêndice mostra como usar o $\text{\LaTeX}$ com a classe `inf-ufg` para formatar teses, dissertações, monografias e relatórios de conclusão de curso, segundo o padrão adotado pelo Instituto de Informática da UFG. Este documento e a classe `inf-ufg` foram, em grande parte, copiados e adaptados da classe `thesisPUC` e do texto de Thomas Lewiner [?] que descreve a sua utilização.

Se você precisar de material de apoio referente ao $\text{\LaTeX}$, veja um dos sites do Comprehensive TEX Archive Network (CTAN) em www.ctan.org. Todos os pacotes podem ser obtidos via FTP <ftp://www.ctan.org> e existem vários servidores em todo o mundo. Eles podem ser encontrados, por exemplo, em <ftp://ctan.tug.org> (EUA), <ftp://ftp.dante.de> (Alemanha), <ftp://ftp.tex.ac.uk> (Reino Unido).

Você pode encontrar muitas informações e dicas na página dos usuários brasileiros de $\text{\LaTeX}$ ($\text{\TeX}$ -BR). Em particular no CTAN, está disponível uma introdução bastante completa em português: CTAN:/tex-archive/info/lshort/portuguese-BR/.

O ambiente Overleaf (<https://pt.overleaf.com>) fornece uma plataforma web completa para processar textos em $\text{\LaTeX}$ sem necessidade de instalar qualquer tipo de software em seu computador. Muitas dicas interessantes sobre $\text{\LaTeX}$ podem ser obtidas em: <https://pt.overleaf.com/learn>.

Todavia, se você quer usar o $\text{\LaTeX}$ em seu computador, verifique em quais sistemas ele está disponível em CTAN:/tex-archive/systems. Em particular para MS Windows, o sistema gratuito MikTeX, disponível no CTAN e no site www.miktex.org tem muitas opções que você poderia precisar para editar o seu texto.

A.2 Descrição da classe inf-ufg

O estilo inf-ufg se integra completamente ao $\text{\LaTeX 2}_{\epsilon}$. O estilo inf-ufg foi desenhado para minimizar a quantidade de texto e de comandos necessários para escrever a sua dissertação. Só é preciso inserir algumas macros no início do seu arquivo $\text{\LaTeX}$, precisando os dados bibliográficos da sua dissertação (por exemplo o seu nome, o título da dissertação...). Em seguida, cada página dos elementos pré-textuais será formatada usando macros ou ambientes específicos. O corpo do texto é editado normalmente. Finalmente, as referências bibliográficas podem ser entradas manualmente (via o comando `\bibitem` do $\text{\LaTeX}$ padrão) ou usando o sistema BiBTeX (muito mais recomendável). Neste caso, os arquivos `inf-ufg.bst` e `abnt-alf.bst` permitem a formatação das referências bibliográficas segundo as normas da UFG.

A.2.1 Opções da classe

Para usar esta classe num documento $\text{\LaTeX 2}_{\epsilon}$, coloque os arquivos `inf-ufg.cls`, `inf-ufg.bst`, `abnt-num.bst`, `atbeginend.sty` e `tocloft.sty` numa pasta onde o compilador $\text{\LaTeX}$ pode achá-lo (normalmente na mesma pasta que seu arquivo `.tex`), e defina-o como o estilo do seu documento. Por exemplo, uma dissertação de mestrado que usa o modelo abnt de citações bibliográficas:

```
\documentclass[dissertacao,abnt]{inf-ufg}
...
\begin{document}
```

As opções da classe são `[tese]` (para tese de doutorado), `[dissertacao]` (para dissertação de mestrado), `[monografia]` (para monografia de curso de especialização) e `[relatorio]` (para relatório final de curso de graduação). Se nenhuma opção for declarada, o documento é considerado como uma dissertação de mestrado. Se a opção `[abnt]` for utilizada, as citações bibliográficas serão geradas conforme definido pelo grupo de trabalho `abnt-tex`. Contudo, o mais recomendável é não utilizar essa opção. Com a opção `[nocolorlinks]` todos os *links* de navegação no texto ficam na cor preta. O ideal é usar esta opção para gerar o arquivo para impressão, pois a qualidade da impressão dos *links* fica superior.

A.2.2 Parâmetros da classe

Os elementos pré-textuais são definidos página por página e dependem da correta definição dos parâmetros listados a seguir (aqueles que contêm um texto/valor padrão não precisam ser definidos, caso atenda a situação do autor do texto que está usando a classe `inf-ufg.cls`):

- \autor : Nome completo do autor da tese, começando pelo apelido (ex.: José da Silva);
- \autorR : Nome completo do autor da tese, começando pelo nome (ex.: da Silva, José);
- \titulo : Título da tese, dissertação, monografia ou relatório de conclusão de curso;
- \subtitulo : Se tiver um subtítulo, use este macro para defini-lo;
- \cidade : A cidade de edição. A cidade padrão é Goiânia.
- \dia : Dia do mês da data de defesa (1–31);
- \mes : Mês da data de defesa (1–12);
- \ano : Ano da data de defesa;
- \universidade : Nome completo da universidade. O nome padrão é Universidade Federal de Goiás;
- \uni : Sigla da universidade. A sigla padrão é UFG;
- \unidade : Nome da unidade acadêmica. O padrão é Instituto de Informática;
- \departamento : Nome do departamento, com maiúscula na primeira letra (para o caso de unidades com mais de um departamento);
- \programa : Nome do programa de pós-graduação, com maiúscula na primeira letra. O padrão é Computação;
- \concentracao : Nome da área de concentração;
- \orientador : Nome completo do orientador, começando pelo apelido;
- \orientadorR : Nome completo do orientador, começando pelo nome;
- \orientadora : Nome completo da orientadora, começando pelo apelido; use este comando e o próximo se for orientadora e nao orientador.
- \orientadoraR : Nome completo do orientadora, começando pelo nome;
- \coorientador : Nome completo do co-orientador, começando pelo apelido;
- \coorientadorR : Nome completo do co-orientador, começando pelo nome;
- \coorientadora : Nome completo da coorientadora, começando pelo apelido; use este comando e o próximo se for coorientadora e nao coorientador.
- \coorientadoraR : Nome completo do coorientadora, começando pelo nome;
- \universidadeco : Nome da universidade do coorientador;
- \unico : Sigla da universidade do coorientador;
- \unidadeco : Nome da unidade acadêmica do coorientador.¹

¹Se não tiver um co-orientador, não defina esses últimos sete parâmetros.

A.2.3 Elementos Pré-Textuais

Os elementos pré-textuais são definidos página por página, conforme descritos a seguir:

capa

`\capa` : Gera o modelo da capa externa do trabalho. Esta página servirá apenas como modelo para a encadernação da versão final do texto. Nenhum dado é necessário.

publicação

`\publica` : Gera a autorização para publicação do trabalho em formato eletrônico e disponibilização do mesmo na biblioteca virtual da UFG.

rosto

`\rosto` : Gera a folha de rosto, a qual é a primeira folha interna do trabalho. Nenhum dado é necessário.

aprovação

`\aprovacao` : ambiente para a reprodução do termo de aprovação da Banca Examinadora da tese ou dissertação.

banca

`\banca` : Entrada para o nome dos examinadores, exceto o(s) orientador(es).
`\profa` : Entrada para o nome das examinadoras, exceto o(s) orientador(es).

direitos

`\direitos` : Macro com 2 argumentos para gerar os direitos autorais, o perfil do aluno e a ficha catalográfica da Biblioteca Central da UFG.

- O primeiro argumento é o Perfil do aluno; e
- O segundo argumento é a lista das palavras-chaves para a Ficha Catalográfica.

dedicatória

`\dedicatoria` : ambiente para escrever a dedicatória. É possível trocar o espaçamento dentro desse ambiente do mesmo jeito que no $\text{\LaTeX}$ padrão.

agradecimentos

`\agradecimentos` : ambiente para escrever os agradecimentos. É possível trocar o espaçamento dentro desse ambiente do mesmo jeito que no $\text{\LaTeX}$ padrão.

resumo

`\chaves` : A lista das palavras chaves, separadas por ‘;’. Deve ser definido antes do ambiente `\resumo`, o qual é usado para escrever o resumo em português.

abstract

`\keys` : A lista das palavras chaves em inglês, separadas por ‘;’. Deve ser definido antes do ambiente `\abstract`, o qual contém 1 argumento e é usado escrever o resumo em inglês. O argumento deve ser o título do trabalho em inglês.

tabelas

`\tabelas` : Macro com 1 argumento opcional para gerar as tabelas. O argumento pode ser:

- nada [] : gera apenas o sumário;
- fig : gera o sumário e uma lista de figuras;
- tab : gera o sumário e uma lista de tabelas;
- alg : gera o sumário e uma lista de algoritmos;
- cod : gera o sumário e uma lista de programas.

Pode-se usar qualquer combinação dessas opções. Por exemplo:

- figtab : gera o sumário e listas de figuras e tabelas,
- figtabcod : gera o sumário e listas de figuras, tabelas e códigos de programas;
- figtabalg : gera o sumário e listas de figuras, tabelas e algoritmos;
- figtabalgcod : gera o sumário e listas de figuras, tabelas, algoritmos e códigos de programas

epígrafe

`\epigrafe` : Macro com 3 argumentos que permite editar um epígrafe. O primeiro argumento é o texto da citação. O segundo argumento é o nome do autor da citação. O terceiro argumento é o título da referência à qual a citação pertence.

A.3 Elementos do texto

A.3.1 Figuras

Rótulos de figuras e tabelas devem ser centralizados se tiverem até uma linha (Figura A.1), caso contrário devem estar justificados e identados em ambas as margens, como mostrado na Figura A.2. Essa formatação já é realizada automaticamente pela classe inf-ufg.

Os compiladores $\text{\LaTeX}$ provêem um mecanismo bastante simples para inclusão de figuras, o que pode ser feito com o auxílio de várias classes auxiliares (as mais comuns são `graphic` e `graphicx`). A classe `inf-ufg` usa o comando `\includegraphics`, da classe `graphicx`, para a inclusão de figuras e não é necessário você colocar a extensão do arquivo neste comando. Por exemplo, para a Figura A.1 os comandos usados foram:

```
\begin{figure}[htb]
\centering
\includegraphics[width=0.40\textwidth]{./fig/exemploFig1}
\caption{Uma figura típica.}
\label{fig:exemploFig1}
\end{figure}
```

Ao se usar o compilador $\text{\LaTeX}$, as figuras podem estar nos formatos *eps* e *ps*. Ao se usar o $\text{PDF}\text{\LaTeX}$, as figuras podem estar nos formatos *png*, *jpg*, *pdf* e *mps*. A classe `graphicx` também pode ser usada para a inclusão de figuras, nos formatos listados, ao se usar o $\text{PDF}\text{\LaTeX}$. Os comandos necessários são os mesmos ao se incluir figuras ao se usar o compilador $\text{\LaTeX}$. O uso do comando `\includegraphics` faz com que $\text{PDF}\text{\LaTeX}$ procure primeiro por figuras com extensão *pdf*, depois *jpg*, depois *mps* e por último *png*. Aqui também não é necessário especificar a extensão do arquivo.

Para a inclusão das figuras A.1 à A.3 os comandos usados, tanto no $\text{\LaTeX}$ quanto no $\text{PDF}\text{\LaTeX}$, seriam os mesmos. É claro que em cada caso devem estar disponíveis as figuras nos formatos suportados por cada compilador. Por exemplo, para a inclusão da Figura A.3 foram usados:

```
\begin{figure}[H]
\centering
\includegraphics[width=0.40\textwidth]{./fig/exemploFig3}
\caption{Figura incluída no texto com a classe graphicx.}
\label{fig:exemploFig3}
\end{figure}
```

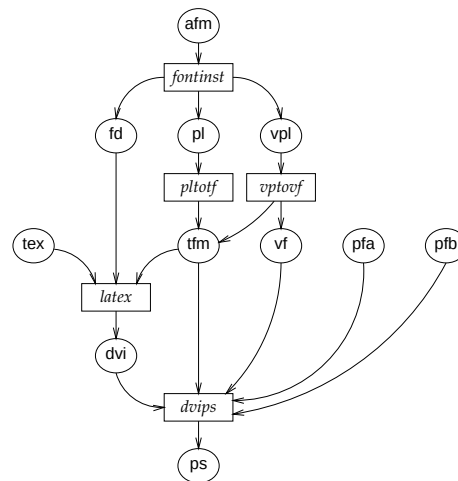


Figura A.1: Uma figura típica.

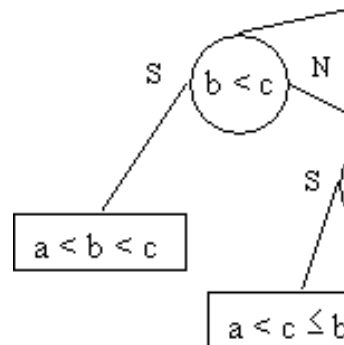


Figura A.2: Esta figura é um exemplo de um rótulo de figura que ocupa mais de uma linha, devendo ser indentado e justificado.

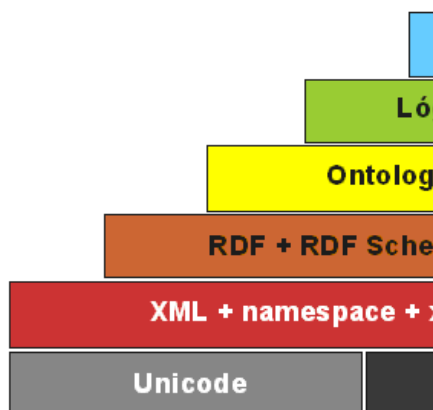
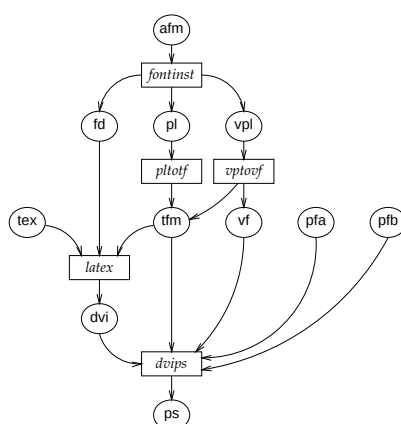


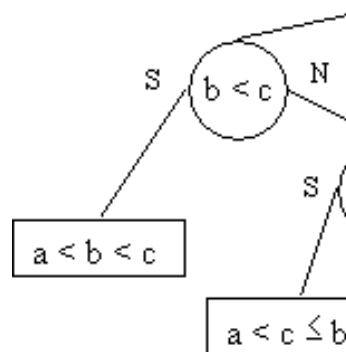
Figura A.3: Figura incluída no texto com a classe `graphicx`.

A.3.2 Subfiguras

A classe `subfigure` pode ser usada para a inclusão de figuras dentro de figuras (consulte a documentação da classe para maiores detalhes). Por exemplo, a Figura A.4 contém duas subfiguras. Estas podem ser referenciadas por rótulos independentes, ou seja, podem ser referenciadas como Figuras A.4(a) e A.4(b) ou Subfiguras (a) e (b).



(a) Primeira subfigura.



(b) Segunda subfigura (um pedaço).

Figura A.4: (a) e (b) representam dois exemplos do uso de subfiguras dentro de uma única figura.

A Figura A.4 foi incluída com os comandos listados a seguir. Observe que há rótulos independentes para cada uma das subfiguras e um rótulo geral para a figura, os quais podem ser todos referenciados.

```
\begin{figure}[h]
\centering
\subfigure[Primeira subfigura.]
{
\includegraphics[width=0.35\textwidth]{./fig/exemploFig1}
\label{subfig:ex1}
} \quad
\subfigure[Segunda subfigura (um pedaço).]
{
\includegraphics[width=0.30\textwidth]{./fig/exemploFig2}
\label{subfig:ex2}
}
\caption{{\subref{subfig:ex1}} e {\subref{subfig:ex2}} representam
dois exemplos do uso de subfiguras dentro de uma única
figura.}
\label{fig:subfiguras}
\end{figure}
```

Caso uma subfiguras não tenha rótulo, para evitar que o apenas o número da mesma apareça na Lista de Figuras, use o comando `\subfigure[] []`. Caso uma subfigura tenha rótulo e deseja-se evitar que a mesma apareça na Lista de Figuras, use o comando `\subfigure[] [Rótulo]`.

A.3.3 Tabelas

Em tabelas, deve-se evitar usar cor de fundo diferente do branco e o uso de linhas grossas ou duplas. Ao relatar dados empíricos, não se deve usar mais dígitos decimais do aqueles que possam ser garantidos pela sua precisão e reprodutibilidade. Rótulos de tabelas devem ser colocados antes das mesmas (veja a Tabela A.1).

A.3.4 Algoritmos

Algoritmos devem ser representados no formato do Algoritmo A.1, que foi descrito com o uso da classe `algorithm2e`. A rigor não é obrigatório o uso dessa classe, contudo o uso da mesma permite que seja gerada automaticamente uma lista de algoritmos logo após o sumário.

Tabela A.1: *Conteúdo do diretório*

Tag	Comprimento	Início		Tag	Comprimento	Início
001	0020	00000		100	0032	00235
003	0004	00020		245	0087	00267
005	0017	00024		246	0036	00354
008	0041	00041		250	0012	00390
010	0024	00082		260	0037	00402
020	0025	00106		300	0029	00439
020	0044	00131		500	0042	00468
040	0018	00175		520	0220	00510
050	0024	00193		650	0033	00730
082	0018	00217		650	0012	00763

Algoritmo A.1: $MSR(A, i, j)$

Entrada: vetor $A[i..j]$, inteiros não negativos i e j .**Saída:** vetor $A[i..j]$ ordenado.

```

1  $n \leftarrow j - i.$ 
2 se  $(n < 4)$  então
3   | Ordene com  $\leq 3$  comparações.
4 senão
5   | Divida  $A$  em  $\lceil \sqrt{n} \rceil$  subvetores de comprimento máximo  $\lfloor \sqrt{n} \rfloor$ .
6   | Aplique  $MSR$  a cada um dos subvetores.
7   | Intercale os subvetores.
8 fim
```

A.3.5 Códigos de Programa

Códigos de programa podem ser importados, mantendo-se a formatação original, conforme se pode ver no exemplo do Código A.1. Este exemplo usa o ambiente `codigo`, definido na classe `inf-ufg`, que permite que uma lista de programas seja gerada automaticamente logo após o sumário.

Código A.1 insertionsort()

```

1 void insertionSort( int* v, int n )
2 {
3     int i    = 0;
4     int j    = 1;
5     int aux = 0;
6
7     while (j < n)
8     {
9         aux = v[j];
10        i   = j - 1;
11        while ((i >= 0) && (v[i] > aux))
12        {
13            v[i + 1] = v[i];
14            i = i - 1;
15        }
16        v[i + 1] = aux;
17        j = j + 1;
18    }
19 }

```

A.3.6 Teoremas, Corolários e Demonstrações

O uso do ambiente theorem permite a escrita de teoremas, como no exemplo a seguir:

```

\begin{theorem}[Pitágoras]
Em todo triângulo retângulo o quadrado do comprimento
da hipotenusa é igual a soma dos quadrados dos
comprimentos dos catetos.
\end{theorem}

```

O resultado é o mostrado a seguir:

Teorema A.1 (Pitágoras) *Em todo triângulo retângulo o quadrado do comprimento da hipotenusa é igual a soma dos quadrados dos comprimentos dos catetos.*

Da mesma forma pode-se usar o ambiente proof para demonstrações de teoremas:

```

\begin{proof}
Para demonstrar o Teorema de Pitágoras \dots
\end{proof}

```


Neste caso, o resultado é:

Prova. Para demonstrar o Teorema de Pitágoras ...

□

Além desses dois ambientes, estão definidos os ambientes `definition` (Definição), `corollary` (Corolário), `lemma` (Lema), `proposition` (Proposição), `comment` (Observação).

A.3.7 Citações Longas

Segundo as normas da ABNT, uma citação longa (mais de 3 linhas) deve seguir uma formação especial. Para tanto foi criado o ambiente `citacao`, o qual é baseado no ambiente de mesmo nome definido pelo grupo ABNTEX [?]:

Uma citação longa (mais de 3 linhas) deve vir em parágrafo separado, com recuo de 4cm da margem esquerda, em fonte menor, sem as aspas [?, 4.4] e com espaçamento simples [?, 5.3]. Uma regra de como fazer citações em geral não é simples. É prudente ler [?] se você optar por fazer uso freqüente de citações. Para satisfazer às exigências tipográficas que a norma pede para citações longas, use o ambiente `citacao`.

Este exemplo de citação longa foi produzido com o uso do ambiente `citacao`, como descrito logo a seguir:

```
\begin{citacao}
Uma citação longa (mais de 3 linhas) deve vir em parágrafo
separado, com recuo de 4cm da margem esquerda, em fonte menor,
sem as aspas \cite[4.4]{NBR10520:2001} e com espaçamento
simples \cite[5.3]{NBR14724:2001}. Uma regra de como fazer
citações em geral não é simples. É prudente ler
\cite{NBR10520:2001} se você optar por fazer uso freqüente
de citações. Para satisfazer às exigências tipográficas que a
norma pede para citações longas, use o ambiente citacao.
\end{citacao}
```

A.3.8 Referências Bibliográficas

Esta seção mostra exemplos de uso de referências bibliográficas com `BIBTEX` e do comando `\cite`. As entradas listadas na página 28 são obtidas com esses recursos. Um grande repositório de referências já em formato `BIBTEX` está disponível em: <http://www.math.utah.edu/beebe/bibliographies.html>.

As referências bibliográficas devem ser não ambíguas e uniformes. Recomenda-se usar números entre colchetes, como por exemplo [?], [?] e [?]. O comando `\nocite` não produz texto, mas permite que a entrada seja incluída nas referências. Por exemplo,

o comando `\nocite{Ber1970}` gera na lista de referências bibliográficas a entrada referente à chave `Ber1970`, mas não inclui nenhuma referência no texto. O comando `\nocite{*}` faz com que todas as entradas do arquivo de dados do `BIBTEX` sejam incluídas nas referências.

Existem vários livros sobre `LATEX`, como [?, ?], embora os mais famosos sejam sem dúvida [?] e [?]. Para converter documentos `LATEX` para HTML veja [?, pg.1–10].